ELECTRIC BUS LOCATION TRACKER SYSTEM

Database Design Document

V 2.0

By

1	Mehroz Ali Khan	NUM-BSCS-2024-34
2	Ali Abbas	NUM-BSCS-2024-06
3	Laiba Taj	NUM-BSCS-2024-31

Department of Computer Sciences
Namal University
Mianwali, Pakistan

Submission Date: 14th June, 2026

REVISION HISTORY

Date	Version	Description
21/06/26	V 2.0	Final implementation: relational schema (3NF), DDL/DML scripts, stored procedures, triggers, GUI integration, and documentation of changes from Milestone 1 & 2.
15/05/26	V 1.0	Initial ERD and conceptual database design (Milestone 2).

Contents

CHAPTER 1 PROJECT OVERVIEW

1.1 INTRODUCTION

The **Electric Bus Location Tracker System** is a mobile-based application for the electric bus transportation network operating in Mianwali, Punjab, Pakistan, under the Punjab Mass Transit Authority (PMA). The system provides real-time tracking of electric buses, management of bus routes and stops, scheduling of driver duties, and generation of operational reports for administrators.

The platform serves three categories of stakeholders:

- **Passengers**: use public-facing screens to search routes, view live bus locations, and check estimated arrival times (no database account required; favourites stored locally).
- **Drivers**: view their assigned duties, start and complete duty cycles, and share their live GPS location while on duty.
- **Administrators**: manage drivers, buses, routes, stops, schedules, duty assignments, and operational reports.

The database forms the backbone of the entire system. It stores user accounts, driver and administrator profiles, bus and route information, stop locations, trip schedules, duty assignments, live GPS location history, generated reports, and a complete audit trail of system activity. This document describes the conceptual, logical, and physical design of that database, along with its implementation through DDL and DML scripts, stored procedures, triggers, and the GUI client that connects to it.

1.2 PROBLEM STATEMENT

Public transportation authorities in Mianwali previously had no centralized digital system to manage bus operations. Passengers had no reliable way to know the real-time location of buses, the routes available, or the expected arrival time at a given stop, leading to wasted waiting time and inconvenience.

On the operational side, administrators managing the electric bus fleet faced difficulties in keeping track of driver duty schedules, bus maintenance and availability status, route and stop information, and historical operational data. Manual record-keeping resulted in data redundancy, scheduling conflicts (e.g., assigning one driver to two overlapping duties, or one bus to two routes at the same time), and an absence of structured reporting.

The core problem addressed by this project is therefore the absence of a normalized, constraint-enforced relational database capable of supporting real-time bus tracking, route and schedule manage-

ment, driver duty lifecycle management, and administrative reporting, while preventing data redundancy and scheduling conflicts through database-level business rules.

1.3 PROJECT OBJECTIVES

The general objective of this project is to design, normalize, and implement a relational database for the Electric Bus Location Tracker System, and to build a GUI client that interacts with this database exclusively through stored procedures, functions, and triggers.

The specific objectives (achieved in Milestone 3) are:

- Convert the approved Entity-Relationship Diagram (ERD) from Milestone 2 into a complete relational schema expressed in Third Normal Form (3NF).
- Identify functional dependencies for every relation and document the normalization steps from 1NF through 3NF.
- Implement the complete schema – twelve base tables, one reporting view, four duty-lifecycle triggers, and twenty stored procedures – using a single `dbDDL.sql` script.
- Populate the database with representative sample data through a `dbDML.sql` script, with a minimum of 20 rows per table (exceeding the requirement).
- Develop a Flutter-based, role-specific GUI (Admin, Driver, and Passenger interfaces) for performing data entry, retrieval, update, and deletion operations.
- Connect the GUI to the database through a Node.js/Express REST API, ensuring that every database operation is performed exclusively through stored procedures, functions, or triggers – never through direct ad-hoc SQL from the client.
- Enforce business rules such as preventing overlapping duty assignments, restricting duty status transitions, and validating account/bus status before assigning duties, through database triggers.

1.4 GITHUB REPOSITORY LINK

- **Source Code & Backend:** https://github.com/MAKDevelopers34/App-Development/tree/main/electric-bus-tracker
- **Documentation & SQL Scripts:** https://github.com/LaibaTaj31/Electric_Buses_Tracker_DB-project.git

CHAPTER 2 CONCEPTUAL DATABASE DESIGN

2.1 ENTITIES

The following entities are central to the Electric Bus Location Tracker System. Associative entities (e.g., route-stop details) are excluded from this list as they are bridge relations.

Table 2.1: List of Entities

Sr. No	Entity Name	Description
01	User	A registered individual acting as either a Driver or an Administrator. Stores account credentials, role, and status.
02	Driver	A subtype of User who operates an electric bus, with license and duty status information.
03	Admin	A subtype of User who manages buses, routes, drivers, schedules, duties, and reports.
04	Bus	An electric bus vehicle in the fleet, identified by registration number, capacity, model, and operational status.
05	Route	A predefined path between a starting point and a destination, including geographic coordinates and distance.
06	Stop	A bus stop location where passengers may board or alight, with geographic coordinates.
07	Schedule	A planned trip linking a Route and a Bus to a departure time, arrival time, and service date.
08	Duty Assignment	A driver's assigned shift, linking a Driver, Bus, and Schedule, created and managed by an Administrator.
09	Bus Location	A timestamped GPS observation recorded while a bus is on an active duty.
10	Report	An operational summary (daily, weekly, or monthly) generated by an Administrator.
11	Audit Log	A system-level record of significant user actions, retained for traceability.

2.2 DATA DICTIONARY

Due to the large number of attributes, only the most representative tables are shown below. The complete data dictionary is available in the project's technical documentation.

Table 2.2: Data Dictionary - users

Sr.	Name	Data Type	Constraint	Description
01	user_id	INT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier for each user
02	username	VARCHAR(50)	NOT NULL, UNIQUE	Login username
03	user_code	VARCHAR(30)	NOT NULL, UNIQUE	Human-readable code (e.g., DRV-001)
04	name	VARCHAR(100)	NOT NULL	Full name
05	email	VARCHAR(100)	NOT NULL, UNIQUE	Email address
06	contact	VARCHAR(20)	NOT NULL	Phone number
07	password_hash	VARCHAR(255)	NOT NULL	Encrypted hash
08	role	ENUM('Driver','Admin')	NOT NULL	User type
09	account_status	ENUM('Active','Inactive')	NOT NULL, DEFAULT 'Active'	Account enabled/disabled
10	login_attempts	INT	NOT NULL, DEFAULT 0	Failed login counter
11	lock_until	DATETIME	NULL	Account lock expiry
12	reset_code	VARCHAR(10)	NULL	Password reset code
13	reset_code_expiry	DATETIME	NULL	Expiry of reset code
14	creation_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
15	deletion_date	DATETIME	NULL	Soft-deletion timestamp

Table 2.3: Data Dictionary - duty_assignments

Sr.	Name	Data Type	Constraint	Description
01	duty_id	INT	AUTO_INCREMENT, PRIMARY KEY	Unique identifier
02	driver_id	INT	NOT NULL, FOREIGN KEY → drivers	Assigned driver
03	bus_id	INT	NOT NULL, FOREIGN KEY → buses	Assigned bus
04	schedule_id	INT	NOT NULL, FOREIGN KEY → schedules	Associated schedule
05	scheduled_date	DATE	NOT NULL	Duty date
06	scheduled_start_time	TIME	NOT NULL	Planned start
07	scheduled_end_time	TIME	NOT NULL	Planned end
08	actual_start_time	DATETIME	NULL	Actual start timestamp
09	actual_end_time	DATETIME	NULL	Actual completion timestamp
10	status	ENUM('Scheduled','In-Progress', 'Completed','Skipped')	NOT NULL, DEFAULT 'Scheduled'	Duty lifecycle state
11	completion_note	TEXT	NULL	Optional note
12	admin_id	INT	NOT NULL, FOREIGN KEY → admins	Creator administrator
13	created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Creation timestamp

2.3 BUSINESS RULES

The following business rules are enforced at the database level through constraints and triggers:

- A User must be either a Driver or an Admin, not both; the `role` attribute enforces this.
- A duty's scheduled end time must always be later than its scheduled start time.
- A Driver cannot be assigned two overlapping duties (same date, overlapping time range) while either duty is `Scheduled` or `In-Progress`.
- A Bus cannot be assigned to two overlapping duties under the same conditions.
- A duty can only be created for a Driver whose linked User account has `account_status = 'Active'`.

- A duty can only be created for a Bus whose `status = 'Active'`.
- A duty's status cannot transition backward from `Completed` to `Scheduled`.
- When a duty's status changes to `In-Progress`, the assigned Driver's status is automatically set to `On-Duty`; when it changes to `Completed` or `Skipped`, the Driver's status reverts to `Available`.
- A User account is locked for 30 minutes after five consecutive failed login attempts.
- A password-reset code is valid for five minutes only.
- Drivers, Admins, Buses, Routes, and Stops are never physically deleted; `deletion_date` or `status = 'Inactive'` is used instead (soft deletion).
- A Route must contain at least one Stop, defined through the `route_stop_details` bridge table with a unique stop order per route.
- Every Report must be linked to exactly one Admin who generated it.
- Every Bus Location record is linked to the Bus, Driver, and Route active at the time of recording, and optionally to the Duty Assignment in progress.
- A duty can be started only within 25 minutes after its scheduled start time (enforced by `sp_start_duty` procedure).

2.4 ENTITY RELATIONSHIP DIAGRAM

The ERD presented below (Figure 1) is the approved diagram from Milestone 2, carried forward as the conceptual foundation for the relational schema. It uses Chen notation: rectangles for entities, diamonds for relationships, ovals for attributes (primary keys underlined), and crow's-foot cardinality markers.

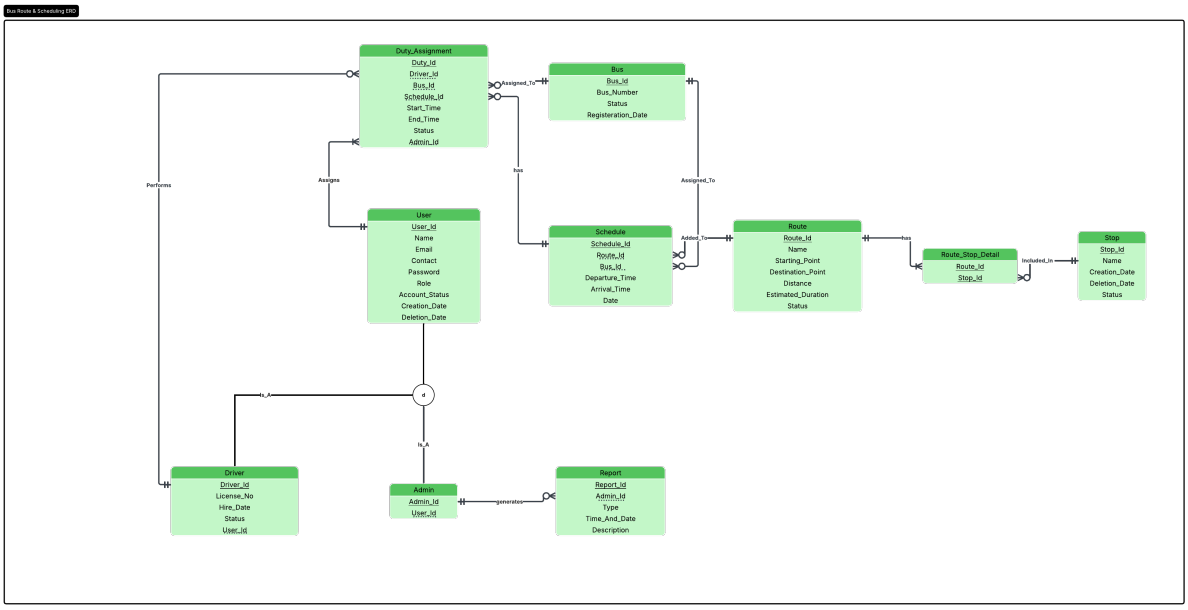

Figure 2.1: Entity-Relationship Diagram for the Electric Bus Location Tracker System (Milestone 2)

CHAPTER 3 LOGICAL DATABASE DESIGN

3.1 RELATIONAL SCHEMA

The ERD was converted into a relational schema following standard ER-to-relation mapping rules. The resulting twelve tables, with primary keys (PK) and foreign keys (FK), are listed below. The complete schema is implemented in `dbDDL.sql`.

Tables

1. **users** (user_id PK, username, user_code, name, email, contact, password_hash, role, account_status, login_attempts, lock_until, reset_code, reset_code_expiry, creation_date, deletion_date)
2. **drivers** (driver_id PK, user_id FK → users, license_no, hire_date, address, status)
3. **admins** (admin_id PK, user_id FK → users)
4. **buses** (bus_id PK, bus_number, capacity, model, status, registration_date, created_at)
5. **routes** (route_id PK, route_code, name, starting_point, start_latitude, start_longitude, destination_point, destination_latitude, destination_longitude, distance, estimated_duration, status, created_at)
6. **stops** (stop_id PK, stop_code, name, latitude, longitude, creation_date, deletion_date, status)
7. **route_stop_details** (route_id, stop_id composite PK, FK → routes, FK → stops, stop_order, distance_from_start, estimated_minutes_from_start)
8. **schedules** (schedule_id PK, route_id FK → routes, bus_id FK → buses, departure_time, arrival_time, service_date, status)
9. **duty_assignments** (duty_id PK, driver_id FK → drivers, bus_id FK → buses, schedule_id FK → schedules, admin_id FK → admins, scheduled_date, scheduled_start_time, scheduled_end_time, actual_start_time, actual_end_time, status, completion_note, created_at)
10. **bus_locations** (location_id PK, bus_id FK → buses, driver_id FK → drivers, route_id FK → routes, duty_id FK → duty_assignments, latitude, longitude, speed, is_active, recorded_at)

11. **reports** (report_id PK, admin_id FK → admins, report_code, type, period_start, period_end, generated_at, total_duties, completed_duties, skipped_duties, total_buses, total_drivers, active_drivers, pdf_path, description)

12. **audit_logs** (audit_id PK, user_id FK → users, action, entity_name, entity_id, details, created_at)

The bridge relation `route_stop_details` resolves the many-to-many relationship between routes and stops. Its composite primary key ensures a stop appears at most once per route, while `stop_order` (unique per route) preserves the visiting sequence.

Referential integrity policy: Foreign keys generally use `ON UPDATE CASCADE` and `ON DELETE RESTRICT` to enforce soft deletion. The exception is `bus_locations.duty_id` which uses `ON DELETE SET NULL`, and `audit_logs.user_id` which uses `ON DELETE SET NULL` to preserve audit history.

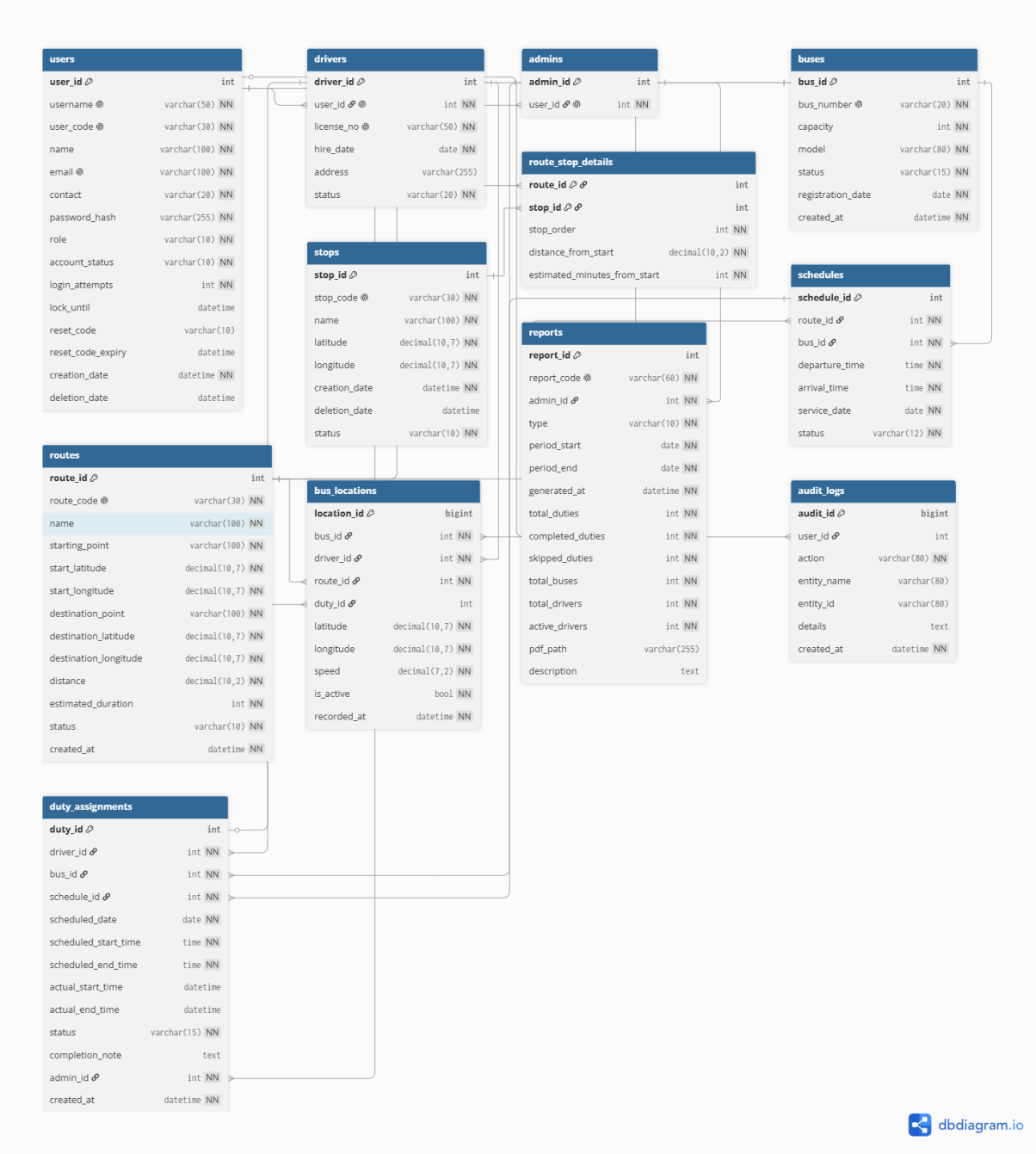

Figure 3.1: Relational Schema Diagram (converted from ERD, generated with dbdiagram.io)

Changes from the Original ERD

The following additions and modifications were made during the conversion from the conceptual ERD (Milestone 2) to the physical relational schema, as reflected in `dbDDL.sql`:

- A new table `audit_logs` was introduced to record all significant user actions (login attempts, duty creation, report generation, etc.).
- The `users` table received additional columns for security and account management: `login_attempts`, `lock_until`, `reset_code`, `reset_code_expiry`, `creation_date`,

and deletion_date.

- Soft deletion support was added to all master tables: deletion_date in users, drivers, stops, and status columns with an 'Inactive' option in buses, routes, and stops.
- Timestamp columns (created_at, creation_date, recorded_at, generated_at) were added to every table for full auditability.
- The many-to-many relationship between routes and stops was explicitly materialized as the route_stop_details bridge table, with additional ordering attributes (stop_order, distance_from_start, estimated_minutes_from_start).
- The schedules table was separated from duty_assignments; duties now reference a schedule via schedule_id to avoid redundancy.
- The reports table was added with snapshot statistics (total_duties, completed_duties, etc.) and a pdf_path column.
- Precise SQL data types were assigned (e.g., DECIMAL(10,7) for coordinates, ENUM for status fields, TIME for scheduled times).
- Foreign key constraints with ON UPDATE CASCADE and appropriate ON DELETE actions (RESTRICT, SET NULL) were defined.

3.2 FUNCTIONAL DEPENDENCIES

For each relation, the functional dependencies (FDs) are listed below.

users

```
user_id → {username, user_code, name, email, contact, password_hash,
role, account_status, login_attempts, lock_until, reset_code, reset_code_
creation_date, deletion_date}
username → user_id
email → user_id
user_code → user_id
```

drivers

```
driver_id → {user_id, license_no, hire_date, address, status}
user_id → driver_id
license_no → driver_id
```

admins

admin_id → user_id
user_id → admin_id

buses

bus_id → {bus_number, capacity, model, status, registration_date,
created_at}
bus_number → bus_id

routes

route_id → {route_code, name, starting_point, start_latitude, start_longi
destination_point, destination_latitude, destination_longitude,
distance, estimated_duration, status, created_at}
route_code → route_id

stops

stop_id → {stop_code, name, latitude, longitude, creation_date,
deletion_date, status}
stop_code → stop_id

route_stop_details

{route_id, stop_id} → {stop_order, distance_from_start, estimated_minutes
{route_id, stop_order} → stop_id (unique ordering per route)

schedules

schedule_id → {route_id, bus_id, departure_time, arrival_time, service_da
status}
{route_id, bus_id, departure_time, service_date} → schedule_id

duty_assignments

duty_id → {driver_id, bus_id, schedule_id, scheduled_date, scheduled_sta
scheduled_end_time, actual_start_time, actual_end_time, status,
completion_note, admin_id, created_at}

bus_locations

`location_id → {bus_id, driver_id, route_id, duty_id, latitude, longitude, speed, is_active, recorded_at}`

reports

`report_id → {admin_id, report_code, type, period_start, period_end, generated_at, total_duties, completed_duties, skipped_duties, total_buses total_drivers, active_drivers, pdf_path, description}`
`report_code → report_id`

audit_logs

`audit_id → {user_id, action, entity_name, entity_id, details, created_at`

3.3 NORMALIZATION

All relations satisfy 1NF (atomic values, no repeating groups). Only one table, `route_stop_details`, has a composite primary key. Its non-key attributes (`stop_order`, `distance_from_start`, `estimated_minutes_from_start`) depend on the whole composite key, because the same stop has different order and distance on different routes – therefore no partial dependency, so 2NF is satisfied.

For 3NF, transitive dependencies were eliminated as follows:

- **Users → Drivers/Admins**: Driver-specific attributes were moved to a separate `drivers` table, and admin-specific attributes to `admins`. This avoids transitive dependency `user_id → role → license_no`.
- **Duty_assignments and schedules**: Route and bus information are kept in `schedules`; `duty_assignments` references `schedule_id` only, eliminating `duty_id → schedule_id → route_id/bus_id`.
- **Routes and route_stop_details**: Stop names and coordinates are stored in `stops`, not duplicated in `route_stop_details`. The bridge table stores only foreign keys and ordering attributes.
- **Reports**: Aggregate counters are snapshot values stored at generation time; they depend only on `report_id` (the snapshot they belong to), so no transitive dependency exists.

After these transformations, every relation is in **3NF**: every non-key attribute depends on the whole primary key and only on the primary key. The complete set of 3NF relations is the twelve tables listed in Section 3.1.

CHAPTER 4 IMPLEMENTATION

4.1 LANGUAGE / FRAMEWORK

The technology stack is summarized in Table 1.

Table 4.1: Technology Stack

Area	**Technology**	**Purpose**
Mobile GUI	Flutter (Dart)	Cross-platform, Android-first GUI for Admin, Driver, and Passenger roles; live maps, GPS publishing, PDF download.
Backend API	Node.js + Express 5	REST routing, JWT authentication, role-based authorization, request validation, business logic, report generation.
Database	MySQL 8 (Amazon RDS)	Relational storage, stored procedures, triggers, views, constraints, indexes.
Authentication	JWT + bcryptjs	Seven-day session tokens and one-way password hashing.
Maps	flutter_map + OpenStreetMap + OSRM	Map rendering and road-following route geometry without a paid API key.
Reports	PDFKit	Server-side generation of daily, weekly, and monthly operational PDF reports.
Email	AWS SES v2	Delivery of password-reset verification codes.
Cloud Hosting	AWS Elastic Beanstalk + RDS	Managed Node.js deployment and managed MySQL database.

4.2 DATABASE CONNECTIVITY

The Flutter GUI never connects to MySQL directly. All database access is mediated by the Node.js/Express REST API, which uses the `mysql2` driver's promise-based connection pool to communicate with the MySQL/RDS instance. The data flow is as follows:

1. The Flutter app sends an HTTP request (with a JWT in the `Authorization` header for protected routes) to the configured API base URL.

2. Express middleware verifies the JWT and checks the user's role (Admin or Driver) before allowing the request to reach the controller.

3. The relevant controller validates the request body and calls the appropriate stored procedure through the connection pool – `CALL sp_xxx(...)`.

4. MySQL executes the stored procedure. Triggers on `duty_assignments` enforce business rules and side effects automatically.

5. The procedure's result set is returned to the controller, which formats it as JSON and sends it back to the Flutter app.

Connection parameters are read from environment variables (`.env`) and are never hard-coded. Per the milestone requirement, the backend controllers issue `CALL` statements to stored procedures for every create, read, update, and delete operation exposed to the GUI.

4.3 STORED PROCEDURES, FUNCTIONS, TRIGGERS, AND VIEW

The database implements **20 stored procedures**, **4 triggers**, and **1 view**. All are documented below.

Table 4.2: Stored Procedures, Triggers, and View

#	Object Name	Type	Description	Input Parameters	Output
1	`sp_get_user_for_login`	Proc.	Retrieves user record for login validation.	p_username, p_user_code	User row
2	`sp_record_login_failure`	Proc.	Increments login attempts and locks account after 5 failures.	p_user_id	None
3	`sp_record_login_success`	Proc.	Resets login attempts and lock_until.	p_user_id	None
4	`sp_get_user_profile`	Proc.	Returns profile information (joined with driver/admin subtype).	p_user_id	Profile row
5	`sp_change_password`	Proc.	Updates password hash after current-password verification.	p_user_id, p_password_hash	None

#	Object Name	Type	Description	Input Parameters	Output
6	sp_save_reset_code	Proc.	Stores a hashed reset code with 5-minute expiry.	p_email, p_reset_code	None
7	sp_reset_password	Proc.	Verifies reset code/expiry and stores new password hash.	p_email, p_reset_code, p_password_hash	Success/ failure status
8	sp_get_routes	Proc.	Returns all active routes with their stop sequence.	None	Route rows
9	sp_create_route	Proc.	Inserts a new route with coordinates and distance.	route_code, name, starting_point, start_lat, start_lng, dest_point, dest_lat, dest_lng, distance, est_duration	New route_id
10	sp_get_drivers	Proc.	Returns all drivers joined with their user account information.	None	Driver rows
11	sp_update_driver	Proc.	Updates a driver's profile and linked user account fields.	p_driver_id, p_name, p_email, p_contact, p_license_no, p_driver_status, p_address	None
12	sp_set_driver_account_status	Proc.	Activates or deactivates a driver's user account.	p_driver_id, p_account_status	None
13	sp_create_duty	Proc.	Creates a new duty assignment; validation triggers run automatically.	p_driver_id, p_bus_id, p_route_id, p_scheduled_date, p_scheduled_start_time, p_scheduled_end_time, p_admin_id	New duty_id

#	Object Name	Type	Description	Input Parameters	Output
14	sp_get_admin_duties	Proc.	Returns all duty assignments with driver, bus, and route details.	None	Duty rows with details
15	sp_get_driver_monthly_duties	Proc.	Returns a driver's duties for a given month and year, plus summary counts.	p_user_id, p_month, p_year	Duty rows + summary
16	sp_start_duty	Proc.	Marks a duty In-Progress and records actual_start_time (within 25 min grace).	p_user_id, p_duty_id	affected_rows
17	sp_complete_duty	Proc.	Marks a duty Completed, records actual_end_time and optional note.	p_user_id, p_duty_id, p_note	affected_rows
18	sp_update_bus_location	Proc.	Inserts a new GPS observation for a bus during an active duty.	p_user_id, p_bus_id, p_route_id, p_duty_id, p_latitude, p_longitude, p_speed	New location_id
19	sp_get_reports	Proc.	Returns all generated reports with admin details (latest 50).	None	Report rows

#	Object Name	Type	Description	Input Parameters	Output
20	sp_create_report	Proc.	Inserts a new operational report with computed statistics and PDF path.	p_admin_id, p_report_code, p_type, p_period_start, p_period_end, p_total_duties, p_completed_duties, p_skipped_duties, p_total_buses, p_total_drivers, p_active_drivers, p_pdf_path, p_description	New report row with admin name
21	trg_duty_before_insert	Trigger	Validates time ordering, driver/bus overlap, and active status before insert.	NEW row	SIGNAL on violation
22	trg_duty_after_insert	Trigger	Sets the driver's status to On-Duty if the new duty is inserted as In-Progress.	NEW row	Updates drivers.status
23	trg_duty_before_update	Trigger	Validates time ordering, blocks Completed→Scheduled transition, and re-checks overlaps.	NEW/OLD row	SIGNAL on violation
24	trg_duty_after_update	Trigger	Synchronizes the driver's status with the duty's new status.	NEW row	Updates drivers.status
25	view_admin_dashboard_stats	View	Aggregates dashboard statistics into a single row.	None	Single row with statistics

4.4 INTERFACES (GUI SCREENSHOTS)

The GUI is implemented as a Flutter mobile application with three role-specific interface groups, all communicating with the Express API. Below are representative screenshots.

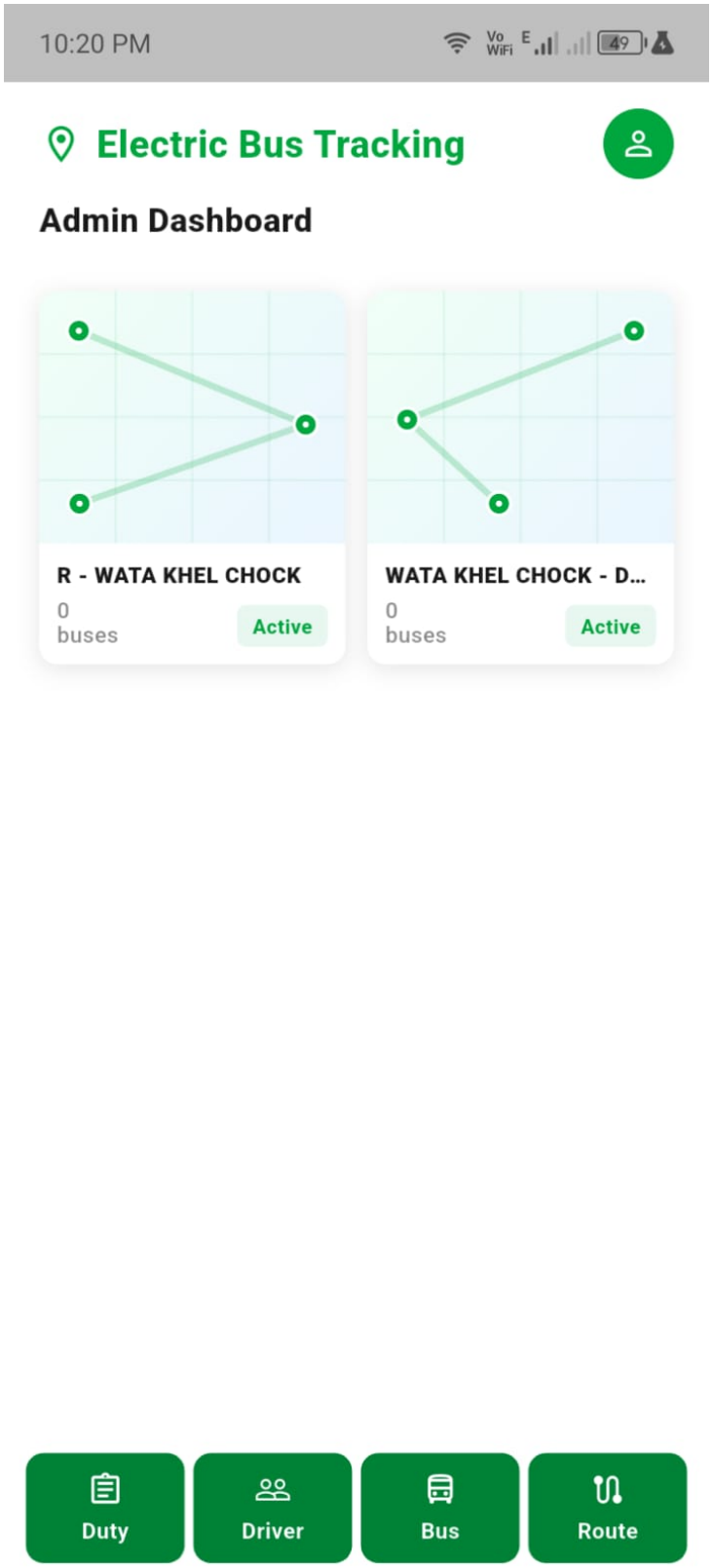

Figure 4.1: Admin Dashboard (statistics and route overview)

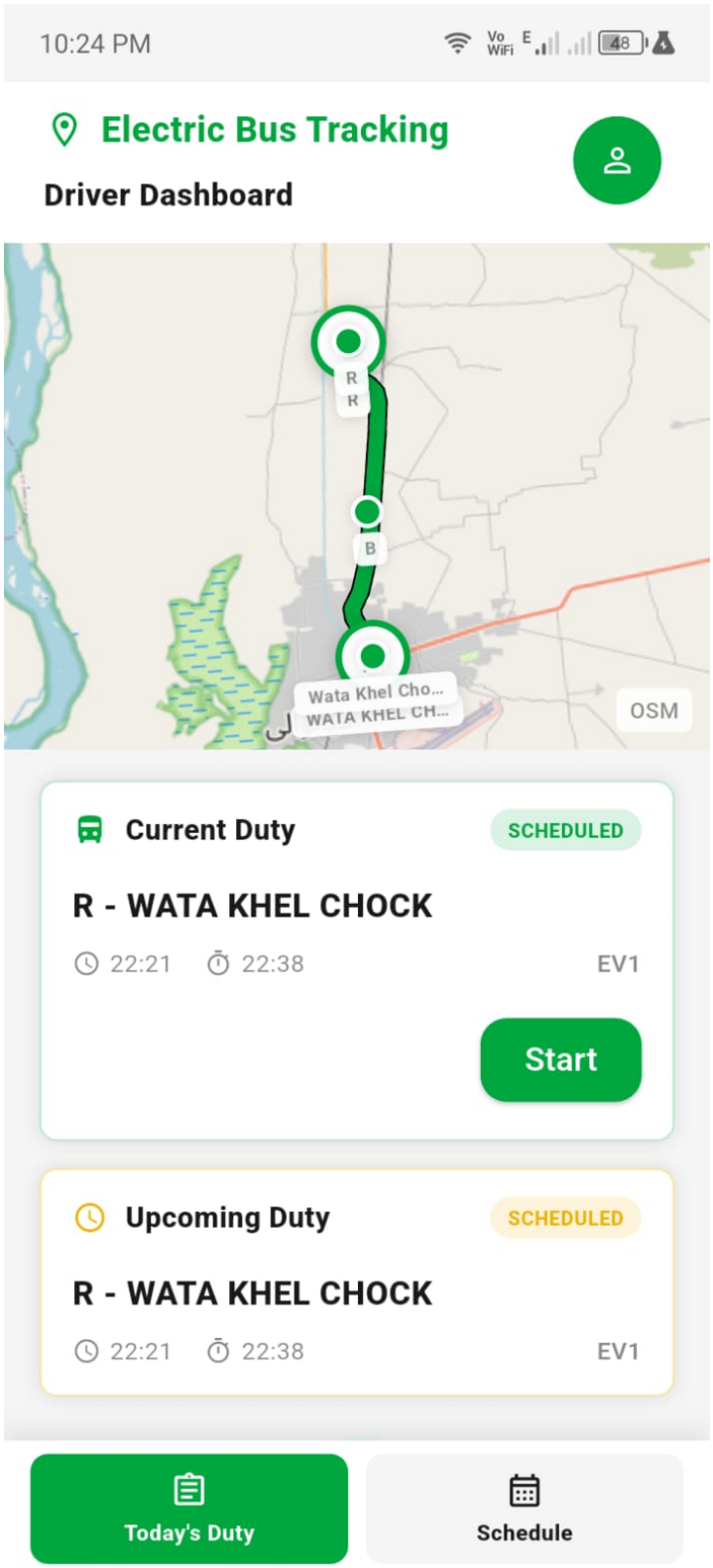

Figure 4.2: Driver Today's Duty Screen (start/completion buttons)

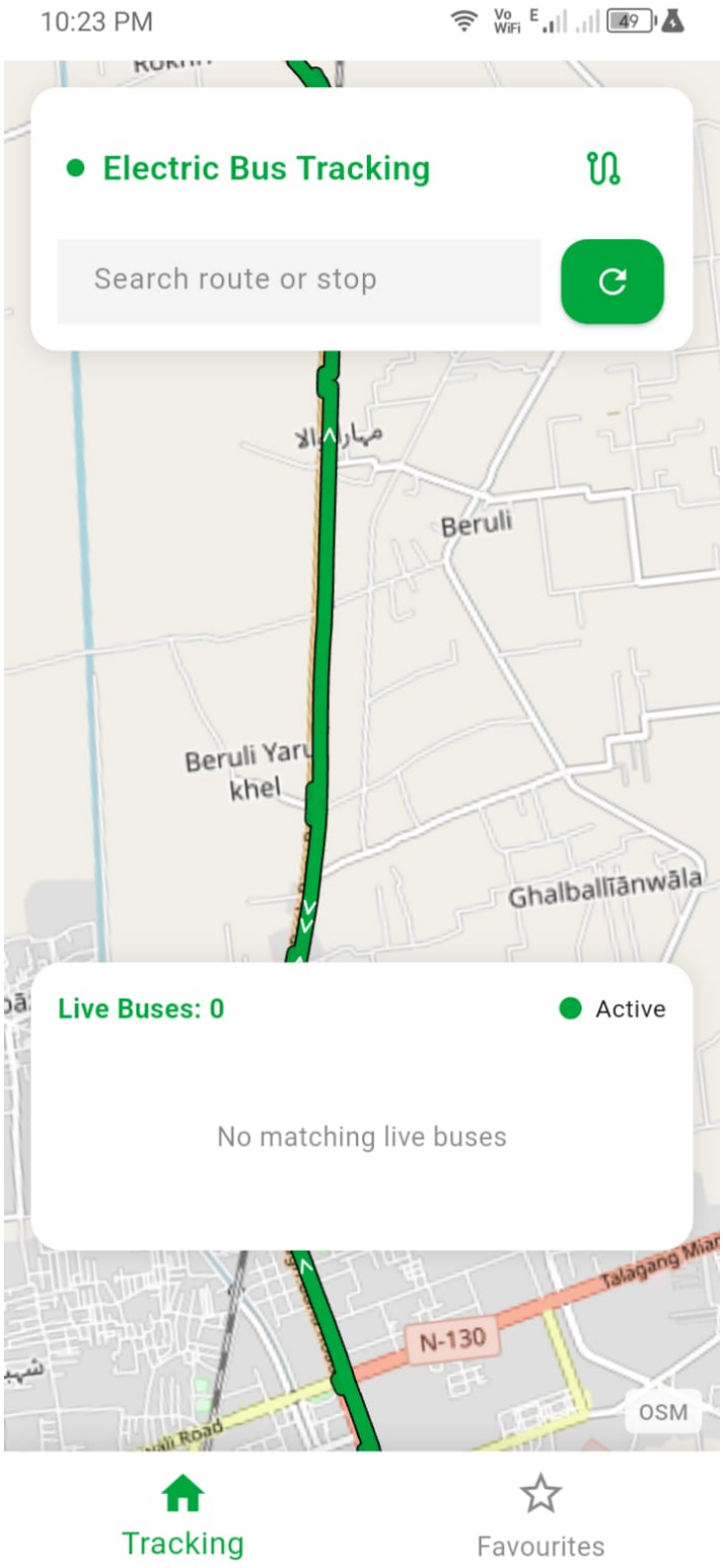

Figure 4.3: Passenger Live Map (bus positions, stops, ETAs)

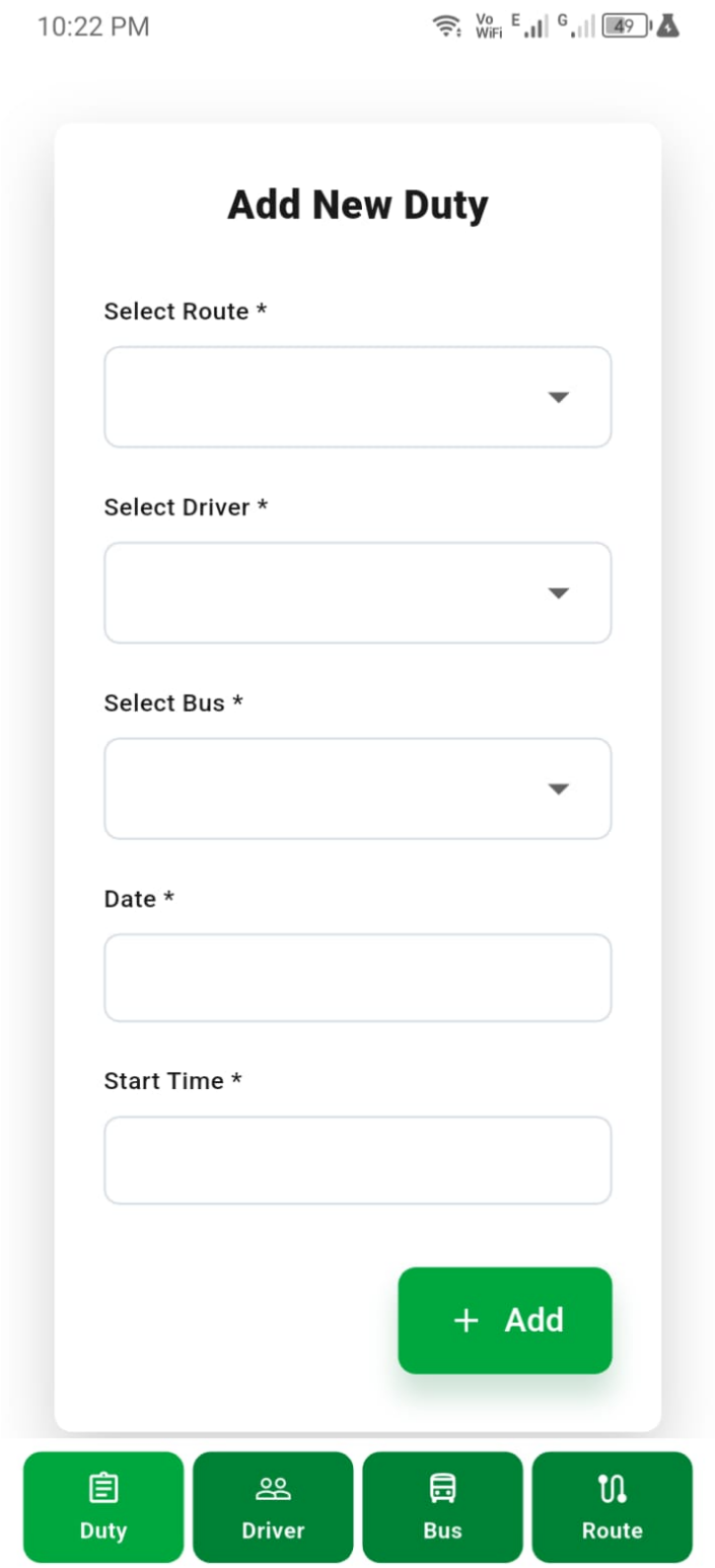

Figure 4.4: Admin – Create Duty Form

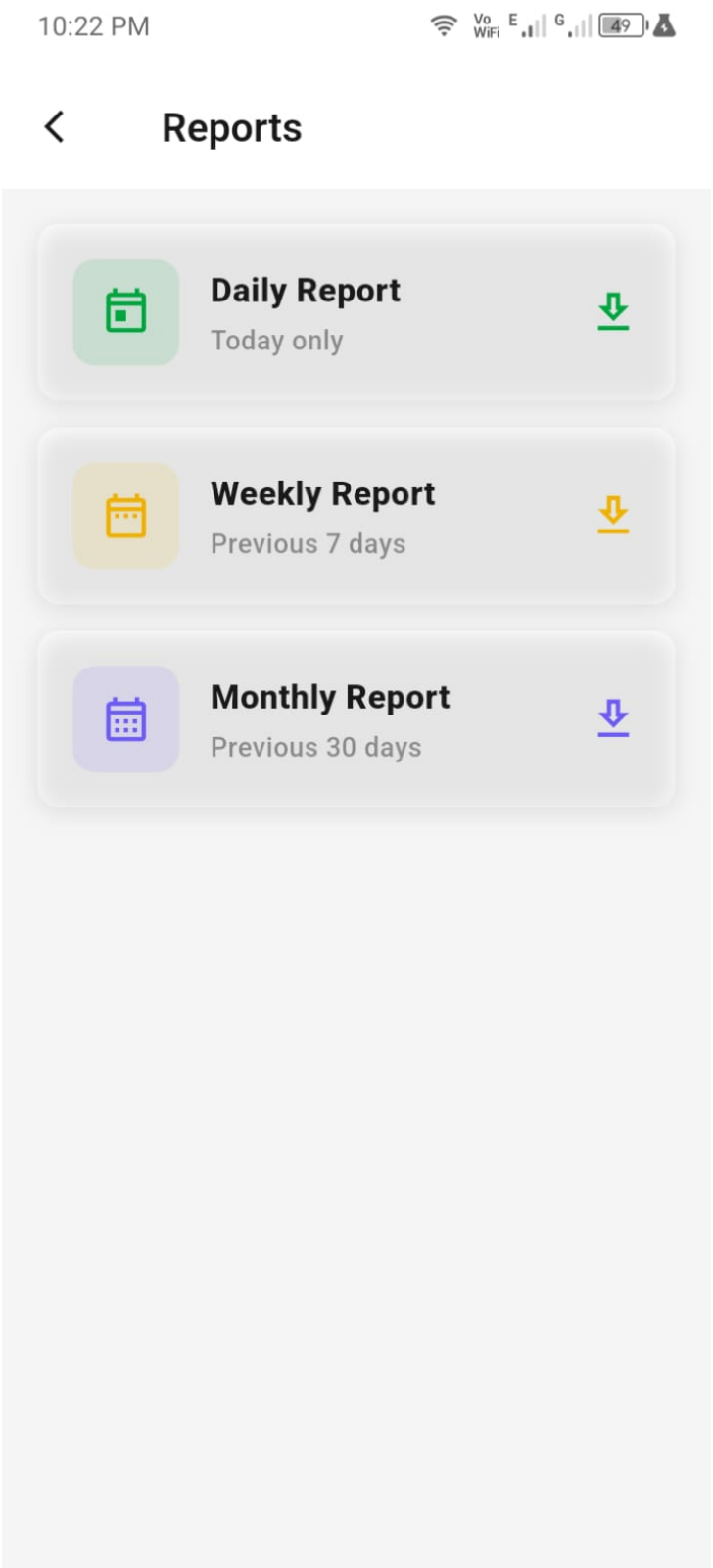

Figure 4.5: Admin – Generate Report (PDF download)

Administrator Interfaces

- **Login and Dashboard**: Secure login screen; dashboard showing route overview and live statistics from the dashboard view.
- **Driver Management**: Screens to register a new driver, list all drivers, edit a driver's profile, and activate/deactivate a driver account.
- **Bus Management**: Screens to add, edit, and deactivate buses, showing capacity, model, and operational status.
- **Route Management**: A map-based screen to create a new route by selecting start, destination, and intermediate stops, with automatic distance and duration calculation.
- **Duty Management**: Screens to create, edit, view, and soft-delete duty assignments, selecting a driver, bus, route/schedule, date, and time window.
- **Reports**: A screen to generate daily/weekly/monthly PDF reports and download them to the device.

Driver Interfaces

- **Duty Views**: Screens listing current, upcoming, completed, pending, and monthly duties.
- **Start/Complete Duty**: A screen to start an eligible duty, during which the app periodically calls the GPS update procedure to publish coordinates, and to complete the duty at the destination.
- **Profile**: View profile information and change or reset the password.

Passenger / Public Interfaces

- **Route and Stop Search**: A search screen that queries active routes and stops and moves the map camera to the selected result.
- **Live Map**: A map screen showing the active route path, direction arrows, live bus positions (sourced from `bus_locations` via GPS endpoints), driver/bus details, and stop ETAs.
- **Favourites**: Locally stored favourite routes/stops on the device (no passenger account or table required).

REFERENCES

[1] P. P. Chen, "The Entity-Relationship Model—Toward a Unified View of Data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9-36, 1976.

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[3] MySQL 8.0 Reference Manual, Oracle Corporation, 2024. [Online]. Available: https://dev.mysql.com/doc/

[4] Express.js Documentation. [Online]. Available: https://expressjs.com/

[5] Flutter Documentation, Google. [Online]. Available: https://flutter.dev/docs

[6] Node.js Documentation. [Online]. Available: https://nodejs.org/docs

[7] OpenStreetMap and OSRM Project Documentation. [Online]. Available: https://project-osrm.org/

[8] Electric Bus Location Tracker - Software Requirements Specification, Namal University, Mianwali, 2024.

[9] Electric Bus Location Tracker - ERD and Conceptual Database Design (CCP Milestone 2), Namal University, Mianwali, 2026.

[10] GitHub Repository Source Code: https://github.com/MAKDevelopers34/App-Development/tree/main/electric-bus-tracker

[11] GitHub Repository Documentation: https://github.com/LaibaTaj31/Electric_Buses_Tracker_DB-project.git

APPENDIX: AI USAGE AND PROMPTS

Table 4.3: AI Tools Usage

Tool	Purpose	Date Accessed
ChatGPT (OpenAI)	Initial guidance on ERD-to-relational mapping and stored procedure structure.	March – May 2026
Claude (Anthropic)	ERD validation (Milestone 2), conversion of approved ERD to 3NF relational schema, functional dependency analysis, normalization documentation, and drafting of this Milestone 3 report from implemented `dbDDL.sql`/`dbDML.sql` scripts.	March – June 2026

Prompts Used

- "Based on this SRS for an electric bus tracking system, help me identify all entities needed for the database design."
- "Validate my supertype-subtype structure for User with Driver and Admin subtypes. What are the correct cardinalities?"
- "Convert the approved ERD from Milestone 2 into a 3NF relational schema, listing primary keys, foreign keys, and the bridge table for the Route-Stop relationship."
- "Given these table definitions from dbDDL.sql, identify the functional dependencies and normalize from 1NF to 3NF, including any transitive dependencies that were resolved."
- "Document the 20 stored procedures and 4 triggers in dbDDL.sql in a table with object name, type, description, input parameters, and output, following the provided Database Design Document template."
- "Generate the Milestone 3 report in the required template structure using the final technical report and dbDDL.sql/dbDML.sql as source material."
- "Produce DBML code for dbdiagram.io without syntax errors."

AI Usage Statement

AI tools were used for guidance, validation, normalization analysis, and formatting/documentation assistance based on the team's own implemented database scripts (`dbDDL.sql`, `dbDML.sql`)

and source code. All schema decisions, business rules, and final content were reviewed and approved by the team.